CLAUSES IN MYsql

MySQL clauses are fundamental components of SQL statements that define how data is retrieved, filtered, grouped, and organized. They act like conditional statements to apply constraints on the data you work with.

1. `WHERE` Clause

Explanation: Filters individual rows based on a condition. It comes before `GROUP BY` and `ORDER BY`.

Ex:

```
SELECT name, price FROM Products WHERE price < 1.0;
```

2. `GROUP BY` Clause

Explanation: Groups rows with identical values in specified columns, often used with aggregate functions on only select command.
Ex: Count products per `productCode`.

1). SELECT productCode, COUNT(*) AS count_per_code FROM Products GROUP BY productCode;

2) SELECT Sales_Manager, Product_Category FROM Sales GROUP BY Sales_Manager, Product_Category;

3. `HAVING` Clause

Explanation: Filters groups created by `GROUP BY` based on aggregate conditions. It is applied after grouping.
Ex: SELECT productCode, COUNT(*) AS count_per_code FROM Products
GROUP BY productCode HAVING count_per_code > 2;

4.`ORDER BY` Clause

Explanation: Sorts the result set by one or more columns in ascending (default) or descending order. Can be used in select, update & delete commands.

Ex: SELECT column1, column2, ...
FROM table_name
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC], ...;

- Mixed Sort Directions**:**

```
SELECT Category, ProductName, UnitPrice
FROM Products
ORDER BY Category ASC, UnitPrice DESC;
```

3. Sorting using Column Aliases

```
SELECT ProductName, (UnitPrice * QuantitySold) AS TotalValue
FROM Products
ORDER BY TotalValue DESC;
```

4. Sorting using Column Position (Index)

```
SELECT ProductName, UnitPrice    FROM Products
ORDER BY 2 DESC; -- Sorts by the second column (UnitPrice)
```

Handling `NULL` Values

```
-- Places NULL salaries at the beginning of an ascending sort
SELECT EmployeeName, Salary
FROM Employees
ORDER BY Salary ASC NULLS FIRST;
```

Sorting by an Expression or Function

```
-- Sorts products based on the product of UnitPrice and QuantitySold
SELECT ProductName, UnitPrice, QuantitySold
FROM Products
ORDER BY (UnitPrice * QuantitySold) DESC;

-- Sorts by the length of the product name
SELECT ProductName
FROM Products
ORDER BY LENGTH(ProductName) ASC;
```

`5.LIMIT` Clause

Explanation: Restricts the number of rows returned. It can specify a count or an offset and count.

Example: Retrieve the first two products ordered by price

```
SELECT name, price FROM Products ORDER BY price LIMIT 2;
```

DISTINCT IN MYSQL:

The MySQL SELECT DISTINCT statement is used to return only unique (different) values within a specified column or set of columns, eliminating duplicate rows from the result set. (only select)

Syntax:

```
SELECT DISTINCT column1, column2, ...
FROM table_name;
```

EX: Using `DISTINCT` on a Single Column

```
SELECT DISTINCT Category FROM Products;
```

2).Using `DISTINCT` on Multiple Columns

```
SELECT DISTINCT Category, Price FROM Products ORDER BY Category, Price ASC;
```

<u>DEFAULT key in mysql :</u>

In MySQL, you can set a default value for a column either when creating a new table or when modifying an existing one**.** This default value is automatically used for any new record where the column's value is not explicitly specified.

Example:

1). CREATE TABLE Orders (
    product_name VARCHAR(255) NOT NULL,
    quantity INT DEFAULT 1,
    order_date DATE DEFAULT CURRENT_DATE()
);

2) **with alter :**

ALTER TABLE Products ALTER COLUMN price SET DEFAULT 0.00;

COMMENTS IN MYSQL:

Style	Syntax	Usage Example
Single-line	`--` (double-dash followed by at least one space or control character)	`SELECT * FROM users; -- This is a single-line comment`
Single-line	`#` (hash symbol)	`SELECT * FROM users; # This is also a single-line comment`
Multi-line	`/* ... */` (C-style block comments)	`/* This comment can span multiple lines */`
Inline	`/* comment */`	`SELECT * /* optional comment */ FROM users;`